HW1 Reflection: Building Research Desk with Codex

Agent-assisted draft for student review. This report describes observable events from the development session. Review the interpretation, add your own experience, and insert the genuine conversation screenshots listed in SCREENSHOTS_TO_ADD.md before submission. Application screenshots and test logs accompany this draft; they do not replace the required screenshots of interaction with the coding agent.

1. Which coding agent did I use, and how?

I used Codex in the desktop application to build an AI Research Assistant. My initial instruction was to read HW1 and complete the work in the solutions directory. Codex read the four-page assignment PDF, identified the five required application workflows and the submission materials, and implemented the project in solutions/hw1.

The agent participated throughout the development process: requirement analysis, architecture selection, backend and frontend implementation, dependency setup, external API integration, debugging, verification, and documentation. The resulting application uses Flask, SQLite, vanilla JavaScript, pypdf, the arXiv API, and an OpenAI-compatible model API. Codex is the development agent; gpt-6-luna is the separate model used inside the finished application.

I supplied the GitHub repository address and configured the API key in a local .env file. I then explicitly selected gpt-6-luna, requested caching, clarified that the spending ceiling was five US dollars, and specified a Nordic minimalist visual style. These were concrete decisions that guided later revisions rather than accepting the first generated version unchanged.

2. Where was the agent particularly useful?

The most useful contribution was connecting the entire workflow. A saved search result is initially only metadata, so the application first downloads and extracts the full PDF before enabling analysis. Uploaded PDFs similarly produce page-numbered text and an editable metadata record. This distinction matters because a summary based only on a paper's title or abstract would not satisfy the assignment.

The agent also handled the less visible work that makes a prototype usable: persistent SQLite records, duplicate handling, clear errors for unreadable PDFs or unavailable providers, and server-side credentials. It created a small set of integration checks and then exercised the actual interface with real arXiv papers. The live test used "Attention Is All You Need" for full-text analysis and "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" for search and saving.

Another useful contribution was making cost control concrete. The final application limits output length, enables the model's prompt cache, saves exact results locally, and reserves a conservative cost before making a request. A repeated question returned an existing saved answer. The live usage log recorded 10,418 cached input tokens, so the caching claim is supported by a provider response rather than only a configuration flag.

3. What was incorrect, incomplete, or unsatisfactory, and how was it addressed?

A functional interface that did not match my design preference

The first interface included promotional text such as "Good research starts with a better question," "Find your next reference. Build your reading list. Go deeper with AI," and a decorative three-card introduction. The features were relevant, but this copy did not fit the concise interface I wanted.

I explicitly requested a Nordic minimalist style and pointed to sentences that should be removed, including "BUILT FOR THE WAY RESEARCH HAPPENS" and "Your library is saved locally · Your ideas, in one place." Codex removed the decorative introduction and sidebar message, shortened headings, and replaced the introductory cards with recently saved papers. The final interface retains neutral backgrounds, a restrained off-white, gray-blue, blue, and navy palette (#F9F7F7, #DBE2EF, #3F72AF, #112D4E), generous spacing, and functional labels. I also rejected a later cream-and-peach palette after seeing it applied to large surfaces; the agent then selected the cooler, mostly neutral palette. This is a clear example of a user correction improving an initially unsatisfactory agent output. The final result is shown in evidence/01-home.jpg; add the real conversation screenshot showing my correction as described in SCREENSHOTS_TO_ADD.md.

A shared-PDF deletion bug

The implementation stores PDFs using a hash of their contents. This deduplicates files, but it means an arXiv download and a manual upload can reference the same physical PDF. The initial deletion route removed that file immediately when deleting either record.

The agent identified this risk during review and added a regression check. The test reproduced the problem: after deleting one record, downloading the other returned HTTP 500 instead of 200 because the file no longer existed. Codex changed deletion to count the remaining references before removing a PDF. The regression then passed. The original failing output and corrected test run are preserved in evidence/shared-pdf-before.txt and evidence/tests-after.txt. This was agent-led debugging; I should not describe it as a fix I personally wrote.

Citation formats that differed from the initial assumption

The first parser recognized only citations shaped like [p. 2]. The real model used combined citations and ranges, including [p. 2, p. 6] and [p. 7–9]. The summary was generated, but the interface initially exposed only a small subset of its supporting pages. This is an example of passing a mocked happy path without covering the variation present in a real model response.

Codex expanded the parser to handle combined citations and bounded page ranges, added a regression check, and rebuilt the source excerpts for already saved answers without making another paid model request. The final application also warns about citations to pages that were not supplied. These checks improve traceability, but they cannot guarantee that a model's interpretation of a cited page is correct.

Environment-specific deployment behavior

A Gunicorn hot reload on macOS triggered an Objective-C fork-related worker crash during integration work. The local demonstration returned to Flask's development server, and the production deployment instructions were scoped to Linux. Docker and reverse-proxy configuration are provided, but the Docker image was not executed in this environment. This distinction prevents deployment configuration from being presented as a verified public deployment.

4. How much inspection, modification, or debugging was required?

The agent generated most of the implementation and performed the automated checks and browser interactions. My direct involvement in this session was setting the destination repository, supplying the local model configuration, choosing the model and budget, and correcting the design. I did not manually rewrite the full codebase, and this report should not imply that I did.

The project needed several review-and-fix cycles after the initial implementation. A working search page did not establish correct persistence, and a successful mocked LLM response did not establish live API compatibility or robust citation parsing. The final check set covers persistence, invalid PDF handling, metadata editing, shared-file deletion, context selection, model request construction, cache reuse, and pre-request budget rejection. Real browser checks separately verified the five required user workflows.

Before submitting, I need to understand the division between app.py, database.py, providers.py, and pdf_tools.py, reproduce the demo, and personally check the displayed answers against the cited pages. The project remains a small single-user prototype. It does not include OCR, separate user accounts, or a semantic retrieval database, and a long-paper summary may use sampled excerpts rather than every page.

5. What did this teach me about communicating with a coding agent?

The session illustrates why concrete constraints are more effective than broad preferences. "Complete HW1" left the agent to choose both the stack and the visual presentation. Naming a model, specifying a dollar ceiling, asking for caching, and identifying exact sentences to remove produced changes that could be checked directly.

It also shows the value of asking for evidence of completion. Cache configuration alone is weaker evidence than a response containing cached-token usage. A successful unit test is different from a real provider call. An attractive page is different from a persistent application. Keeping these distinctions visible made the project's status easier to assess.

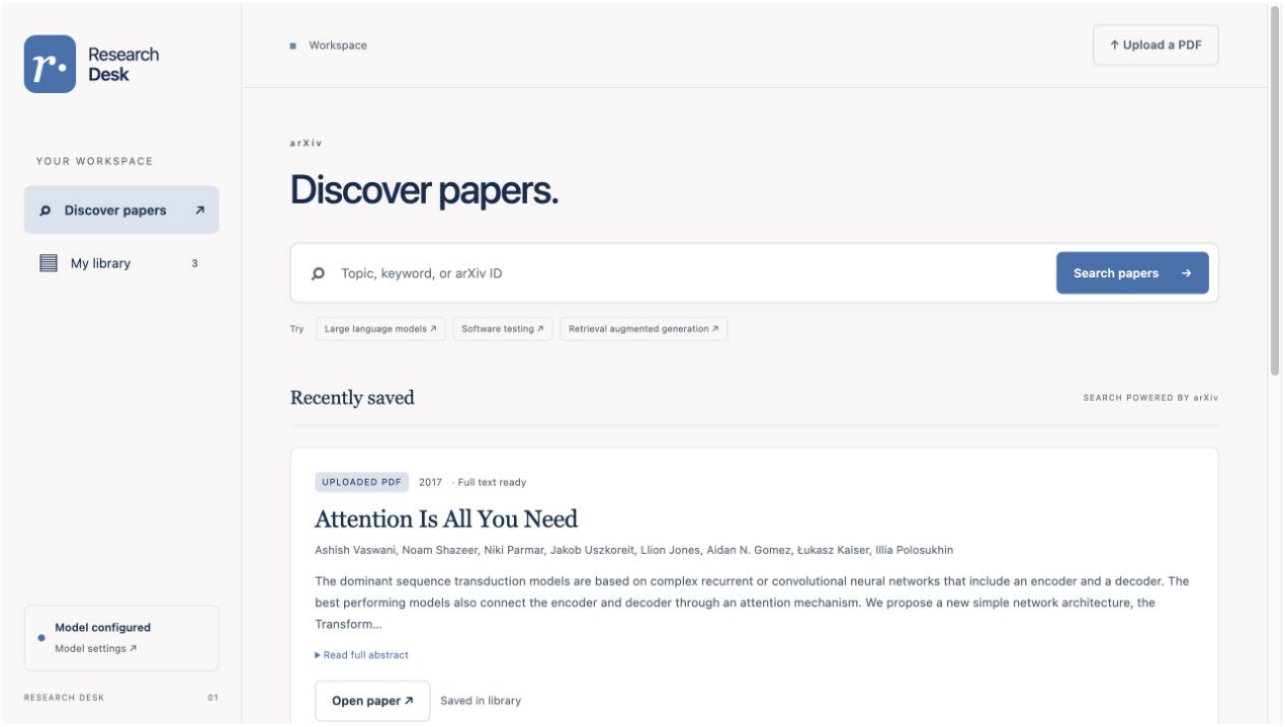
The appropriate role of the agent was to accelerate implementation and investigation while leaving final responsibility with me. The most useful future improvement would be to state acceptance criteria earlier: a minimal interface, actual PDF text for analysis, explicit evidence for claims, server-side secrets, a spending limit, and a repeatable demonstration. Those criteria make both delegation and review more precise.

Supporting evidence and disclosure

- Application screenshots: evidence/01-home.jpg through evidence/08-evidence.jpg.
- Original regression failure: evidence/shared-pdf-before.txt.
- Final check output: evidence/tests-after.txt.
- Real API usage and cache evidence: evidence/model-usage.json.
- This reflection was drafted with Codex and must be reviewed and personalized by the student.
- Genuine screenshots of the conversation still need to be supplied by the student. The automation tool did not permit capturing the Codex application itself; no substitute conversation screenshots were fabricated.

Application evidence

These screenshots show the running application. They are not substitutes for the required coding-agent conversation screenshots.



Final interface with the accepted off-white, gray-blue, and navy palette.